DITA Architect v0.5 Project Plan

Author: Rafael Santos, Engineering Manager **Date:** 2026-03-10 **Status:** DRAFT — FOR TEAM REVIEW **Milestone Target:** Internal daily-use reliability

---

1. Executive Summary

v0.5 is not a feature milestone. It is a correctness and reliability milestone. The bar for shipping is: anyone on this team can open the tool in the morning, author DITA content across multiple tabs, sync with Heretto, and close their browser without wondering if they lost something. That's it.

What v0.5 is not: a refactored codebase, a beautiful UX, or a production-ready product. We are not shipping to external users. We are not solving accessibility in full. We are not standing up a backend or CI that would satisfy a larger organization. Security is explicitly out of scope.

What v0.5 is: the first tagged version. A stake in the ground that says "this works well enough for us to trust daily." That means the data-loss bugs are gone, the multi-tab correctness issues are fixed, the round-trip sync path is tested, and the worst papercut UX issues are addressed so demos don't embarrass us.

One constraint is non-negotiable: round-trip serialization must be covered by tests before we tag v0.5. I said this in the kickoff brief and I am saying it again here. Everything else on this list is negotiable. That one is not.

---

2. Critical Path Items

These must ship. The v0.5 tag does not go on without them. Ordered by risk to the user — specifically, risk of silent data loss.

CP-1: Fix the `nodeOriginMap` Global Singleton

Owner: Anna (lead), Jamie (review) **Rationale:** Flagged by Anna (Finding 1, Critical), Jamie (Finding 3b, highest-priority bug risk), Elena (Summary point 3), and raised in my own kickoff brief. Four independent reviewers landed on the same issue. This is the most dangerous thing in the codebase.

The `nodeOriginMap` is a module-level singleton. There is exactly one instance for the entire application. When a second tab opens, its `parseXmlToLexical` call wipes and repopulates the map. If any background `OnChangePlugin` event fires from another tab's editor in that window, the serializer operates on a corrupted or empty map. The result is body element tags silently replaced by `<p>` tags on the next serialize. That is semantic data loss with no error, no warning, and no recovery path.

Acceptance Criteria:

- `nodeOriginMap` is no longer a module-level export
- Each `SyncManager` instance owns its own map, passed explicitly to `parseXmlToLexical` and `serializeLexicalToXml`
- `ShortdescPlugin` receives the map through props or context rather than importing the global (Anna Finding 8, Jamie Finding 3b)
- Opening two tabs with different content and switching between them does not corrupt either document's structure
- A test demonstrates correct isolation between two simultaneous parse cycles

Blocks: CP-4 (round-trip tests depend on correct map behavior), CP-2 (the XML cache fix should land in the same refactor pass)

CP-2: Fix the Module-Level XML Meta Cache

Owner: Anna (lead) **Rationale:** Anna (Finding 2, Critical), Jamie (Finding 5a), Elena (Logic Correctness, Blocking). Same class of problem as CP-1. The serializer's `_cachedXmlSource` / `_cachedXmlMeta` are module-level singletons keyed on raw XML strings. Two tabs with identical XML content share a cache entry. The stored `Document` object is never released — a memory leak that grows with usage.

Acceptance Criteria:

- XML meta cache is owned by each `SyncManager` instance, not the module
- Two tabs with identical XML content parse and serialize independently without cache interference
- Cache is released when its owning `SyncManager` unmounts

Blocks: CP-4

CP-3: Fix the `patchBodyChildren` Off-by-One Bug

Owner: Elena (lead), Jamie (review) **Rationale:** Elena (Logic Correctness, Blocking). This is the highest-severity confirmed logic bug in the codebase. When two consecutive type mismatches occur, `lexIdx` advances unconditionally, causing the second DOM child to be processed against the wrong Lexical node. The result is silent content corruption — wrong element types in the serialized output. This is not a theoretical edge case; it is a structural flaw in the core patching algorithm.

Acceptance Criteria:

- The off-by-one is corrected with backtracking or look-ahead logic
- Test cases covering consecutive type-mismatch scenarios are added to the sync layer test suite
- Verified by Elena via code review

Blocks: CP-4

CP-4: Round-Trip Serialization Test Suite

Owner: Taylor (lead), Anna (architecture consult), Elena (type consult) **Rationale:** My mandate from the kickoff brief. Taylor (Gaps 1 and 2, Critical) confirmed that `sync/serializeLexicalToXml.ts` (1,019 lines) and `sync/parseXmlToLexical.ts` (354 lines) have zero test coverage. Zero. This is 1,373 lines of the most complex and most consequential code in the project. Taylor's summary says it plainly: "If it's not tested, it doesn't work."

This work cannot begin until CP-1, CP-2, and CP-3 are resolved — tests built on broken foundations certify broken behavior.

Acceptance Criteria:

- Integration-level round-trip tests: DITA XML in → Lexical parse → XML serialization → compare to original
- Tests must cover: heading elements, paragraph content, lists, tables, `<shortdesc>`, inline formatting (bold, italic, underline), `<codeph>`, image references, ph/ref elements, and opaque/unrecognized elements
- `patchInlineNodes` and `patchBodyChildren` have unit tests covering edge cases including consecutive mismatches (the CP-3 bug)
- `DitaPhRefNode.getTextContent()` and `TrackedDeletionNode.getTextContent()` overrides are tested (Taylor Gap 8)
- Tests run cleanly via `npx vitest run`
- Tests are co-located in `tests/sync/` following existing project conventions

Blocks: v0.5 tag

CP-5: Fix `savedXmlRef` Mutation Inside `setTabs` Updaters

Owner: Elena (lead) **Rationale:** Elena (Type Safety, Blocking), Jamie (Finding 2b). `target.savedXmlRef.current = content` is executed inside a `setTabs` functional updater. This directly mutates a property on a React state object, violating React's immutability contract. In React 19 with concurrent features, this is a correctness hazard — the mutated value may be read before the render that produced it completes. This is also the mechanism that makes `Tab.savedXmlRef` the fragile ref-inside-state pattern Jamie identified.

Acceptance Criteria:

- Mutation of refs inside `setTabs` updaters is eliminated
- The pattern for tracking saved XML state is documented so the team doesn't reintroduce it
- Elena signs off

Blocks: CP-4 (tests should validate the corrected state management behavior)

CP-6: Add `test` Script to `package.json` and Fix Vitest Config

Owner: Marcus (lead), Taylor (verify) **Rationale:** Marcus (Recommendations 1, 5). There is no `test` npm script. There is no `setupFiles` for jest-dom matchers. `globals: true` is set in vitest config without the matching `types` entry in tsconfig. `@testing-library/react` and `@vitest/coverage-v8` are installed and unused. You can't mandate tests if running them requires knowing the incantation.

Acceptance Criteria:

- `npm test` runs `vitest run`
- `vitest.config.ts` has `setupFiles` configured for jest-dom matchers
- `tsconfig.json` includes `"vitest/globals"` in `types`
- Coverage reporting works (`npm run coverage` or equivalent)
- Marcus verifies the full toolchain runs clean from a fresh checkout

Blocks: CP-4 (Taylor can't write the round-trip tests against a broken test infra)

CP-7: Add `.env*` and `coverage/` to `.gitignore`

Owner: Marcus **Rationale:** Marcus (DevOps Review, .gitignore section). `.env*` is missing from `.gitignore` on a project that handles Heretto credentials. This is the kind of thing that creates an uncomfortable conversation with legal someday. Takes five minutes. There is no acceptable reason to defer this.

Acceptance Criteria:

- `.env*` added to `.gitignore`
- `coverage/` added to `.gitignore`
- `*.tsbuildinfo` added to `.gitignore`

Blocks: Nothing, but it must ship in v0.5

---

3. High Priority Items

Important for daily use quality. Should ship with v0.5 but will not hold the tag if we're genuinely close.

HP-1: Add `aria-label` to All Icon-Only Toolbar Buttons

Owner: Maya (spec), Jamie (implementation) **Rationale:** Maya (Accessibility, 19 specific unlabeled interactive elements). Rafael (kickoff brief). This is the minimum accessibility work that keeps us from being embarrassed in a demo to anyone who cares about inclusive design. It is also the easiest accessibility win on the list — `aria-label` attributes on existing elements, no architecture required.

Acceptance Criteria:

- Every icon-only button in `Toolbar.tsx`, the tab bar, and the Heretto context bar has an `aria-label`
- Maya reviews and signs off on label text
- No `aria-label` text longer than 5 words

HP-2: Add Confirmation Dialog to `Clear Contents`

Owner: Maya (spec), Jamie (implementation) **Rationale:** Maya (Content Design, Clear Contents button). One misclick destroys the document. Tab close has a confirm modal. Clear contents fires immediately and then toasts `toast.error()` for a successful action. The inconsistency is actively confusing. This is a one-afternoon fix.

Acceptance Criteria:

- `Clear Contents` shows a confirmation modal before executing
- Confirmation modal matches the existing modal style in the codebase
- The toast severity is corrected from `toast.error()` to `toast()`

HP-3: Add Error Surfaces to Heretto Polling and Search

Owner: Jamie (lead) **Rationale:** Jamie (Section 6, Error Handling Gaps), Elena (Error Handling, Blocking). `herettoSearch` swallows all fetch errors and returns `[]` silently. The polling loop surfaces nothing. Users working with Heretto have no idea when background operations are failing. For a CMS integration tool, silent failures in the CMS integration path are not acceptable for daily use.

Acceptance Criteria:

- `herettoSearch` logs errors and surfaces a user-facing indication when folder fetches fail (partial results are labeled as such)
- The polling loop surfaces errors — at minimum a toast with discriminated messaging (credential error vs. network timeout vs. server error)
- `doHerettoRefresh` logs the actual error before showing the toast

HP-4: Move `formatXml` to `lib/xml-utils.ts`

Owner: Anna (lead), Jamie (review) **Rationale:** Anna (Finding 5, Separation of Concerns), Jamie (Summary item 3), Elena (cross-team finding). `useHerettoCms.ts` and `lib/xml-utils.ts` both import `formatXml` from `components/MonacoDitaEditor.tsx`. A business logic hook and a pure utility library importing from a UI component is an inverted dependency that will cause pain in any future decomposition of `MonacoDitaEditor`.

Acceptance Criteria:

- `formatXml` is extracted to `lib/xml-utils.ts` or a new `lib/xml-format.ts`
- `MonacoDitaEditor.tsx` imports from the lib, not the other way around
- `useHerettoCms.ts` imports from the lib
- Existing tests for `formatXml` pass without modification

HP-5: Add `npm run lint` via ESLint (Minimal Config)

Owner: Marcus **Rationale:** Marcus (Missing Infrastructure table). No linter. No formatter. We cannot write tests to cover `any` types if the tooling doesn't tell us when we introduce new ones. A minimal ESLint config with TypeScript and React rules takes a morning to set up and catches the class of issues Elena spent her entire review documenting.

Acceptance Criteria:

- ESLint installed with `@typescript-eslint` and `eslint-plugin-react-hooks`

- `"lint": "eslint src"` added to `package.json` scripts
- `npm run lint` passes on the current codebase (or all existing violations are explicitly suppressed with `// eslint-disable` comments and filed as tech debt)

HP-6: Fix `TrackedChangesPlugin` Deletion Descriptor Collision

Owner: Elena (lead) **Rationale:** Elena (Logic Correctness, Blocking). The deletion descriptor uses `::` as a separator. If any tracked text content contains `::`, the descriptor parsing collides. This is a data corruption bug in the tracked changes feature. Not as systemic as CP-3, but a genuine correctness failure in a feature the team uses.

Acceptance Criteria:

- Deletion descriptor format uses a separator that cannot appear in DITA text content
- Or the parser is updated to handle the collision correctly
- Test case covering `::` in tracked text content

HP-7: Remove or Silence Synchronous Formatting Toasts

Owner: Maya (spec), Jamie (implementation) **Rationale:** Maya (Content Design, Toolbar button labels). Toasting `Formatted as Heading 1` or `Toggled Bold` after every formatting operation is noise. It degrades the tool's feel during a demo and trains users to ignore toasts — which means they miss the ones that matter. Maya identified 10 specific synchronous formatting toasts to remove.

Acceptance Criteria:

- Synchronous formatting operation toasts are removed
- Async and error toasts remain
- Maya approves the final list of toasts to keep vs. remove

---

4. Nice to Have

Good ideas. v0.6 material.

- **`window.prompt()` replacement for Insert Link** — Maya (Finding 7), Jamie (Summary item 8). A proper modal for URL input. Correct thing to do, not urgent enough for v0.5.
- **Modal keyboard accessibility** — Maya (Accessibility section). `role="dialog"`, `aria-modal`, focus trapping. Real work, real value. v0.6.
- **Dropdown keyboard navigation** — Maya (Accessibility section). Arrow keys, Escape, `aria-haspopup`. Same timeline.
- **Decompose `dita-architect.tsx` into modal components** — Jamie (Section 1), Anna (Finding 3). Extract `ConfirmModal`, `HerettoStatusModal`, `HerrettoBrowserModal`, etc. This is the right architecture, and it will make the file testable. But it is a significant refactor that carries real regression risk. Worth doing properly, not rushed.
- **`useHerettoCms` sub-hook split** — Anna (Finding 4). 39 return values from one hook is bad API design. Splitting into `useHerettoNav`, `useHerettoSave`, `useHerettoSearch` would help. Same regression caution applies.
- **Heretto polling backoff** — Anna (Finding 6). Exponential backoff with jitter. Important for production; tolerable for daily internal use.
- **TTL on `herettoFolderCache`** — Anna (Finding 11), Jamie (Section 7). The BFS cache grows unbounded. An LRU cap or TTL is the right fix. Not causing data loss; causing stale search results.
- **`noUnusedLocals` / `noUnusedParameters` in tsconfig** — Marcus. Good hygiene, but enabling it cold against a 1,700-line god component will generate noise.
- **Remove `@lexical/yjs` unused dependency** — Marcus. One line. Not critical.
- **`DitaTopicType` defined in two places** — Elena. Consolidate into `types/`. Drift risk, not immediate harm.

- **Replace inline hover handlers with CSS** — Jamie (Section 2), Maya (Design Consistency). 30-40 places. Real cleanup, no user impact.

---

5. Out of Scope for v0.5

Explicitly deferred. These are not forgotten — they are decisions, not omissions.

Backend / production deploy — Anna (Phase 5, v1.0 Prerequisite). The Vite dev server is the proxy. That is fine for internal use. Designing a production backend is a separate project.

Security hardening — Rafael (kickoff brief). Explicitly out of scope. The Heretto credential storage approach is known. We will revisit before any external shipment.

CI/CD pipeline — Marcus (Recommendation 8). We should have one. We don't have the infrastructure scaffolding to do it right in this cycle. Marcus can set up a GitHub Actions skeleton, but full pipeline automation is v0.6.

`MonacoDitaEditor.tsx` decomposition — Anna (Finding 5). 868 lines, five concerns. The correct split is: Monaco React wrapper, XML formatter, DITA content model, completion provider, auto-close logic, syntax themes. This is a worthwhile but lengthy refactor. Doing it wrong introduces regressions. Deferring.

`patchSectionFromHeading` half-implementation — Jamie (Finding 5d). May silently lose section body content. This is flagged and needs a proper fix, but it requires understanding the full section-conversion use case. Filing as a known issue for v0.6.

Flesch-Kincaid performance — Jamie (Section 7). `BottomToolbar` runs the calculation on every edit for all tabs. Real performance hit on large documents. Debounce or idle scheduling is the fix. Deferred.

TrackedChangesPlugin O(n²) LCS — Jamie (Finding 3d). No throttle, no debounce. Will degrade on large documents. A real problem for heavy tracked-change users, tolerable for v0.5 internal use.

Full accessibility compliance — Maya identified 19 specific unlabeled interactive elements, zero keyboard-accessible dropdowns, zero modal focus management. We are doing the aria-label pass (HP-1) and the Clear Contents guard (HP-2). Full WCAG compliance is a longer arc.

Dead code removal — Elena (Section 7). `DitaPhRefNode.__originalText`, `herettoSearchAbortRef`, `readingLevel`, `TRACKED_CLASSES`, `ClearEditorPlugin`. No user impact. Clean up on a quiet afternoon, not in the v0.5 crunch.

---

6. Dependencies and Ordering

The sequencing matters. Here is the order in which work must happen.

Week 1:

1. **Marcus** does CP-6 (test script + vitest config) and CP-7 (.gitignore) in parallel — these are fast and unblock everything else
2. **Anna** starts CP-1 (`nodeOriginMap` per-instance) — this is the highest-risk item and the longest lead time
3. **Elena** starts CP-3 (`patchBodyChildren` off-by-one) and CP-5 (`savedXmlRef` mutation)
4. **Anna** does HP-4 (`formatXml` extraction) in parallel with CP-1 since it's a straightforward move

Week 2:

5. **Anna** finishes CP-1 and CP-2 (XML meta cache — these are coupled, should land together)
6. **Elena** finishes CP-3 and CP-5, delivers to Taylor for test review
7. **Taylor** begins CP-4 (round-trip tests) once CP-1, CP-2, CP-3, and CP-5 are merged and CP-6 infra is stable
8. **Jamie** works HP-1 (aria-labels), HP-2 (Clear Contents guard), HP-3 (Heretto error surfaces) in parallel
9. **Maya** reviews HP-1 label text and HP-2 modal copy

10. **Marcus** does HP-5 (ESLint) once the CP items are stable — don't introduce lint gates while major refactors are in flight

Final gate:

11. Taylor runs the full test suite. All 196 existing tests pass. New round-trip tests pass. Sync layer tests pass. No regressions.
12. Marcus verifies clean build: `tsc --noEmit`, `npm test`, `vite build` all exit 0.
13. v0.5 tag.

---

7. Test Gates

Nothing ships without these. This is not aspirational — these are the conditions under which the v0.5 tag is applied.

Gate 1 — Existing tests still pass `npm test` runs all 196 existing tests green. No regressions from any of the CP or HP work.

Gate 2 — Round-trip serialization tests pass (Rafael's mandate) DITA XML in → Lexical parse → XML serialization → compare to original. Must cover: headings, paragraphs, lists, tables, shortdesc, inline formatting (bold/italic/underline), codeph, images, ph/ref elements, opaque elements, and documents with processing instructions. This is Taylor's primary deliverable for v0.5.

Gate 3 — `patchBodyChildren` edge cases tested Consecutive type-mismatch scenarios explicitly covered. Validates the CP-3 fix.

Gate 4 — Custom node text content overrides tested `DitaPhRefNode.getTextContent()` and `TrackedDeletionNode.getTextContent()` have tests confirming the correct return values. These feed directly into serializer output.

Gate 5 — Multi-tab isolation test Opening two tabs, switching between them, and serializing both produces the correct XML for each tab with no cross-contamination. Validates CP-1 and CP-2.

Gate 6 — Clean build `npx tsc --noEmit` exits 0. `npm test` exits 0. `npx vite build` exits 0. Marcus owns this verification.

---

8. Risk Register

Risk	Likelihood	Impact	Mitigation
CP-1 refactor (`nodeOriginMap`) introduces regressions in existing serialization behavior	Medium	High	Taylor runs the existing 196 tests after each stage. Anna pairs with Jamie on review. Gate 5 catches multi-tab regressions specifically.
Round-trip tests (CP-4) reveal additional correctness bugs that weren't on the list	Medium	Medium	This is actually a feature of having tests. Any new bugs found become HP items or v0.6 items depending on severity. We do not delay the tag for bugs we discover in tests if they are not data-loss bugs.
`patchBodyChildren` fix (CP-3) is deeper than a one-line backtrack	Medium	Medium	Elena escalates to Anna if the fix requires restructuring the walk algorithm. Pair on it. Two days max before we reassess scope.
`savedXmlRef` fix (CP-5) interacts with the `setTabs` patterns in `useHerettoCms` in unexpected ways	Medium	Medium	Elena maps the full mutation surface before changing anything. Jamie reviews.

Taylor's CP-4 test writing reveals that the sync layer functions are not extractable without a Lexical editor instance	High	High	Taylor and Anna meet on day one of week 2 to assess extractability. If pure unit tests are blocked by Lexical instance requirements, we fall back to integration-level tests with a lightweight Lexical environment. Taylor flagged this strategy in her review.
Vite build failures discovered during Marcus's Gate 6 check	Low	Low	Marcus was blocked from running `vite build` during the review. We may have unknown build issues. Marcus runs this in week 1 before any refactoring is done to establish a clean baseline.
HP items slip and cause schedule pressure on the CP items	Low	Medium	HP items do not block the v0.5 tag. If we hit week 2 and CP-4 is in progress, HP items are dropped to v0.6 without negotiation.

---

9. Team Assignments

Playing to what each person does well. No one is working outside their zone.

Anna Sidorova (Staff Engineer) CP-1 (nodeOriginMap fix, lead), CP-2 (XML meta cache, lead), HP-4 (formatXml extraction, lead) Anna wrote the definitive architectural findings. She understands the sync layer's invariant structure better than anyone and is the right person to refactor the singleton patterns without breaking them. She is also the reviewer on CP-3 and CP-5 to catch any cross-system effects.

Jamie Okafor (Senior Frontend Engineer) HP-1 (aria-labels, implementation), HP-2 (Clear Contents guard, implementation), HP-3 (Heretto error surfaces, lead), HP-7 (formatting toasts, implementation) Jamie's review was the most implementation-specific — section headers and code references throughout. He has the tactical React and component-level instincts for the HP items. He reviews Anna's CP-1 work for React-pattern correctness.

Elena Vasquez (Principal Engineer) CP-3 (patchBodyChildren off-by-one, lead), CP-5 (savedXmlRef mutation, lead), HP-6 (TrackedChangesPlugin descriptor collision, lead) Elena found the deepest logic bugs. The off-by-one in `patchBodyChildren` and the `::` collision in `TrackedChangesPlugin` both require the kind of careful algorithmic analysis that is her strength. She signs off on her own fixes and also serves as reviewer on CP-4 to validate that Taylor's tests are testing the right behavior.

Maya Chen (UX Engineer) HP-1 (aria-label text, spec and review), HP-2 (Clear Contents modal, spec), HP-7 (formatting toast list, spec) Maya produced 28 specific copy recommendations and a complete accessibility audit. Her job in v0.5 is to spec the UX changes clearly enough that Jamie can implement them without iteration. She defines what the aria-labels say, what the Clear Contents dialog says, and which toasts survive the cut. She is not implementing.

Marcus Wren (Senior DevOps Engineer) CP-6 (test script + vitest config, lead), CP-7 (.gitignore, lead), HP-5 (ESLint, lead), Gate 6 (clean build verification) Marcus's work is the unblocking work. CP-6 and CP-7 happen in the first two days and unlock Taylor. HP-5 happens after the major refactors stabilize. Marcus also establishes a baseline vite build result in week 1 before refactoring begins, so we know what we're starting from. He owns the Gate 6 sign-off.

Taylor Brooks (QA Lead) CP-4 (round-trip tests, lead) Taylor's review told us exactly what is missing and exactly how to fix it. She gets to build it. CP-4 is her primary and only v0.5 deliverable — it is significant enough work that she should not be pulled onto anything else. She pairs with Anna and Elena on the architectural questions about what can be tested without a live Lexical instance. She also verifies that the existing 196 tests still pass after each CP item lands.

Rafael Santos (Engineering Manager) Coordination, prioritization, review sign-off on CP items, and the Gate 2 check on the round-trip tests. I will read every test Taylor writes for the sync layer before we tag. That is not optional.

Appendix: Confirmed Bugs vs. Architectural Issues

For clarity: the CP items above include a mix of data-loss bugs and architectural correctness issues. The distinction matters for how we communicate status.

Confirmed data-loss bugs (must fix before v0.5):

- CP-1: `nodeOriginMap` global singleton → silent element-type corruption on tab switch
- CP-3: `patchBodyChildren` off-by-one → silent content corruption on consecutive type mismatches
- HP-6: `TrackedChangesPlugin` `::` collision → descriptor corruption on matching text

Architectural correctness issues (fix now to prevent future data-loss bugs):

- CP-2: XML meta cache global → cache poisoning between tabs
- CP-5: `savedXmlRef` mutation inside `setTabs` → React state contract violation

Test coverage gaps (not bugs, but unverified behavior):

- CP-4: 1,373 lines of sync code with zero tests

Everything else is quality, maintainability, or UX. Important. Not v0.5 blocking.

Rafael Santos, Engineering Manager, Cybergymnastics Inc. 2026-03-10